*flight system

Step 1: Architectural Style: Modular Monolith

This project follows a Modular Monolith architecture with a package-by-business-capability structure.

Each root package represents a main business capability, while sub-packages represent related features. For example, payment and refund are placed under billing, booking and passenger under reservation, and flight, airport, and seat under catalog.

---

step 2: Oracle Database Connection

The first step was configuring the Spring Boot project to connect with Oracle Database.

The required dependencies were added, including Oracle JDBC Driver and Spring Data JPA. Then, the database connection settings were configured in application.properties using the Oracle URL, username, and password.

- **app properties :**

spring.datasource.url=jdbc:oracle:thin:@//localhost:1521/xepdb1 → Database connection URL

spring.datasource.username=hr → Database username

spring.datasource.password=123 → Database password

spring.datasource.driver-class-name=oracle.jdbc.OracleDriver → Oracle JDBC driver

spring.jpa.hibernate.ddl-auto=update → Auto update tables

spring.jpa.show-sql=true → Show SQL queries

spring.jpa.properties.hibernate.format_sql=true → Format SQL output

---

step 3: Product Backlog

The implementation backlog is ordered by dependency

The system follows this **implementation order**:

Foundation
→ Identity and Security
→ Administration
→ Catalog
→ Reservation
→ Billing
→ Ticketing
→ Notification
→ Reporting and Audit

This order is used because protected operations depend on users, roles, authentication, and authorization. Booking depends on flights and seats. Payment depends on booking. Ticket generation depends on successful payment and confirmed booking. Refund depends on paid or cancelled bookings. Reports and audit depend on existing operational data.

Phase 0 — Project Foundation

Goal

Prepare the backend project structure, database connection, and shared response/error handling before implementing business features.

Features

F0.1 Create Final Package Structure

Module: All modules

common, identity, administration, catalog, reservation, billing, ticketing, notification, reporting

Done When

- The package structure is clean.
- Root packages represent business modules.
- Small features are grouped under their parent module.

F0.2 Connect Spring Boot to Oracle Database

Module: Project configuration

Done When

- Oracle connection properties are configured.
- The application can connect to Oracle successfully.
- Hibernate can create/update tables when entities are added.

F0.3 Add Common API Response Format

Module: common.response

Purpose

Create a standard response wrapper for success, error, validation, and paginated responses.

The API Contract defines a consistent response format for success, errors, validation errors, and paginated data.

Expected Classes

ApiResponse<T>
ErrorResponse
PageResponse<T>

Done When

- All future APIs can return the same response structure.
- Success responses include success, message, and data.
- Error responses include success, message, errorCode, and details.

F0.4 Add Global Exception Handling

Module: common.exception

Expected Classes

GlobalExceptionHandler
BusinessException
ResourceNotFoundException
DuplicateResourceException
UnauthorizedException
ForbiddenException

Done When

- Validation errors return a clean response.
- Duplicate email returns 409 Conflict.
- Not found resources return 404 Not Found.
- Forbidden operations return 403 Forbidden.

Phase 1 — Identity and Access Management

Goal

Implement users, authentication, login, JWT, and role-based access control.

This phase must come first because the API Contract separates public endpoints from protected endpoints, and protected endpoints require a JWT token in the Authorization: Bearer <access_token> header. The main roles are CUSTOMER, AIRLINE_ADMIN, and SUPER_ADMIN.

F1.1 User Model

Module: identity.user

Purpose

Create the base user model used by Customers, Airline Admins, and Super Admins.

Expected Classes

User
UserRole
UserStatus
UserRepository

Relevant Business Rules:
- All system users are represented by one User model.
- A user must have one role: CUSTOMER, AIRLINE_ADMIN, or SUPER_ADMIN.
- A user must have one status: ACTIVE, BLOCKED, or DEACTIVATED.
- Passwords must never be stored as plain text; the User model stores passwordHash only.
- Email must be unique because users log in using their email.
- Deactivated users must not access protected endpoints.
- User accounts should be deactivated instead of permanently deleted where historical data must be preserved.

Implementation Notes:
- UserRole and UserStatus are implemented now as enums.
- email uniqueness is added now at the entity/database level and later checked in the service.
- passwordHash is added now as a field, but password hashing is implemented later in Register/Login.

- preventing deactivated users from accessing protected APIs is implemented later in AuthService and Security.

Done When

- USERS table is created in Oracle.
- UserRole enum is added.
- UserStatus enum is added.
- User entity contains the required fields.
- UserRepository can find users by email.
- UserRepository can check whether an email already exists.

F1.2 Register Customer

Module: identity.auth + identity.user

Endpoint: POST /api/auth/register

Access: Public

Actor: Customer

Purpose: Allow a new customer to create an account through the public registration endpoint.

Expected Classes:

- RegisterCustomerRequest
- RegisterCustomerResponse
- AuthController
- AuthService

Relevant Business Rules:

- Customer registration is a public operation and does not require a JWT token.

- Only Customer accounts can be created through the public registration endpoint.

- The registration request must not allow the client to choose the user role.

- The system must always assign the role CUSTOMER to users created from this endpoint.

- The system must always assign the status ACTIVE to newly registered customers.

- Airline Admin accounts cannot register directly from the public registration endpoint.

- Airline Admin accounts are created only by a Super Admin.

- Super Admin accounts are not created through the public customer registration endpoint.

- Email must be unique because users log in using their email address.

- The password must never be stored as plain text.

- The raw password received in the request must be hashed before saving the user.

- The user must be saved in the USERS table.

- The response must not return the password or passwordHash.

- If the email already exists, the API must return 409 Conflict.

- If the request data is invalid, the API must return 400 Bad Request.

F1.3 Login

Module: identity.auth

Endpoint: POST /api/auth/login

Access: Public

Actor: Customer, Airline Admin, Super Admin

Purpose: Authenticate a user using email and password. If the credentials are valid, the backend returns a JWT access token and the authenticated user data.

Expected Classes

• LoginRequest

• LoginResponse

• AuthService.login()

Relevant Business Rules

• Login is public and does not require a JWT token.

• Login is available for all system roles: CUSTOMER, AIRLINE_ADMIN, and SUPER_ADMIN.

• The system must authenticate users using email and password.

• Invalid email or password must not reveal which value is wrong.

• Passwords are compared using the stored passwordHash, not plain text.

• Deactivated users must not log in or access protected APIs.

• Blocked users should be prevented from logging in if the BLOCKED status is used.

• Successful login must return an access token with tokenType Bearer.

• Successful login must return user data needed by the frontend: userId, name, email, role, userStatus, and airlineId when applicable.

Implementation Notes

• Email lookup is handled through UserRepository.findByEmail().

• Password matching is handled using PasswordEncoder.matches(rawPassword, passwordHash).

• Account status is checked in AuthService before generating the JWT token.

• JWT generation is called after credentials and status are valid.

• The login response must not return password or passwordHash.

Done When

• POST /api/auth/login works.

• Invalid email or password returns 401 Unauthorized.

• Deactivated account returns 403 Forbidden.

• Successful login returns accessToken and tokenType Bearer.

• Successful login returns the authenticated user data.

• The response does not expose password or passwordHash.

F1.4 JWT Generation and Validation

Module: identity.security

Purpose: Generate JWT tokens after successful login and validate JWT tokens before allowing access to protected APIs.

Expected Classes

• JwtService

• JwtAuthenticationFilter

• SecurityConfig

• CustomUserPrincipal

Relevant Business Rules

• All protected endpoints require authentication.

• The JWT token must be sent in the Authorization header using the format: Bearer <access_token>.

• The system must validate authentication tokens before allowing access to protected APIs.

• The system must apply role-based access control using CUSTOMER, AIRLINE_ADMIN, and SUPER_ADMIN roles.

• JWT must identify the authenticated user and their role.

• Invalid, missing, or expired tokens must result in 401 Unauthorized.

• Authenticated users without the required role must receive 403 Forbidden.

• Deactivated users must not access protected endpoints, even if they have a previously issued token.

Implementation Notes

• SecurityConfig defines public endpoints such as register, login, airport list, flight search, and flight details.

• JwtAuthenticationFilter extracts the Bearer token from the Authorization header.

• JwtService validates the token and extracts the subject/user identity and role claims.

• CustomUserPrincipal represents the authenticated user inside the security context.

• Role restrictions are applied either in SecurityConfig or method-level security where needed.

Done When

• Login returns a valid JWT access token.

• The frontend can send Authorization: Bearer <access_token>.

• Protected endpoints cannot be accessed without a valid token.

• Invalid token returns 401 Unauthorized.

• User with wrong role receives 403 Forbidden.

• Public endpoints remain accessible without token.

F1.5 Current User Profile

Module: identity.auth + identity.user

Endpoints:

• GET /api/auth/me

• PUT /api/auth/me

• PUT /api/auth/change-password

Access: Authenticated user

Actor: Customer, Airline Admin, Super Admin

Purpose: Allow any logged-in user to view and manage their own profile and change their password.

Expected Classes

• CurrentUserResponse

• UpdateProfileRequest

• ChangePasswordRequest

• AuthController

• UserService or AuthService methods

Relevant Business Rules

• Current user profile endpoints require a valid JWT token.

• Any logged-in user can view their own profile regardless of role.

• Any logged-in user can update only their own profile data.

• Profile update should allow only editable fields such as firstName, lastName, and phone.

• Users must not update their role, status, email, or airlineId through the profile update endpoint.

• Change password requires the correct old password.

• The new password must be hashed before saving.

• The response must not expose password or passwordHash.

Implementation Notes

• The current user is read from the authenticated principal/security context.

• Profile update belongs to user data, but the endpoint is under auth because it uses the current authenticated user.

• Change password uses PasswordEncoder.matches() for the old password and PasswordEncoder.encode() for the new password.

Done When

• Logged-in user can view their profile.

• Logged-in user can update firstName, lastName, and phone.

• Logged-in user cannot change role, userStatus, or airlineId through profile

update.

• Logged-in user can change password after entering the correct old password.

• Invalid old password returns a clear error response.

• All endpoints require JWT authentication.

Phase 2 — Administration

Goal: Implement Super Admin operations for managing airlines, Airline Admin accounts, and customer accounts.

F2.1 Create Initial Super Admin Account

Module: identity.user

Purpose : Create the first Super Admin account so protected administration APIs can be tested.

Relevant Business Rules

• Super Admin is a User with role SUPER_ADMIN.

• Super Admin accounts are not created through the public customer registration endpoint.

• Super Admin operations must be restricted to authorized Super Admin accounts only.

• The Super Admin password must be stored as passwordHash, not plain text.

• The initial Super Admin account should have status ACTIVE.

Implementation Notes

• Use a temporary CommandLineRunner seed or a manual Oracle insert during development.

• Remove or protect development seed logic before production-like use.

• The seeded account should follow the same password hashing rules as normal users.

Done When

• A SUPER_ADMIN user exists in the USERS table.

• The Super Admin can log in successfully.

• The Super Admin receives a valid JWT token.

• The Super Admin can access SUPER_ADMIN protected endpoints.

F2.2 Airline Management

Module: administration.airline

Endpoints:

• POST /api/airlines

• PUT /api/airlines/{airlineId}

• PATCH /api/airlines/{airlineId}/deactivate

• GET /api/airlines

Access: SUPER_ADMIN

Purpose Allow Super Admin to create, update, deactivate, and view airlines.

Expected Classes

• Airline

• AirlineStatus

• AirlineRepository

• AirlineService

• AirlineController

• CreateAirlineRequest

• UpdateAirlineRequest

• AirlineResponse

Relevant Business Rules

• Only SUPER_ADMIN can create, update, deactivate, and list airlines.

• Airlines are deactivated instead of permanently deleted.

• Deactivated airline status must become INACTIVE.

• An inactive airline cannot have new flights added later.

• Old flights, bookings, payments, tickets, and reports related to a deactivated airline must remain stored.

• Airline data should be unique where needed according to business identifiers such as airline name or code if added.

• Sensitive admin actions such as deactivating an airline should be audit logged later.

Implementation Notes

• Create and update request DTOs validate required fields such as airlineName and contactNumber if used.

• Deactivate should update airlineStatus to INACTIVE, not delete the row.

• Audit logging can be added when reporting.audit is implemented, or called directly if AuditService exists earlier.

Done When

• Super Admin can create an airline.

• Super Admin can update airline information.

• Super Admin can deactivate an airline.

• Deactivated airline status becomes INACTIVE.

• Non-Super Admin users cannot access airline management endpoints.

F2.3 Airline Admin Management

Module: administration.airlineadmin

Endpoints:

• POST /api/airline-admins

• PUT /api/airline-admins/{adminId}

• PATCH /api/airline-admins/{adminId}/deactivate

• GET /api/airline-admins

Access: SUPER_ADMIN

Purpose: Allow Super Admin to create and manage Airline Admin accounts.

Expected Classes

• CreateAirlineAdminRequest

• UpdateAirlineAdminRequest

• AirlineAdminResponse

• AirlineAdminService

• AirlineAdminController

Important Design Decision

Airline Admin is stored as a User with role AIRLINE_ADMIN and an assigned airlineId. It is not stored as a separate AirlineAdmin entity.

Relevant Business Rules

• Only SUPER_ADMIN can create, update, deactivate, and list Airline Admin accounts.

• Airline Admin accounts cannot register from the public registration endpoint.

• Airline Admin accounts are created only by Super Admin.

• Each Airline Admin must be assigned to one airline.

• The assigned airline must exist.

• A new Airline Admin user must have role AIRLINE_ADMIN.

• A new Airline Admin user must have status ACTIVE.

• Airline Admin password must be hashed before saving.

• Airline Admins can manage only data related to their assigned airline.

• Deactivation must update userStatus to DEACTIVATED instead of deleting the user.

Implementation Notes

• AirlineAdminService uses UserRepository to create/update users with role AIRLINE_ADMIN.

• AirlineAdminService validates the airline through AirlineRepository or AirlineService.

• The response must not expose password or passwordHash.

• Deactivate should preserve historical data related to the admin actions and airline.

Done When

• Super Admin can create an Airline Admin.

• Created user has role AIRLINE_ADMIN.

• Created user has status ACTIVE.

• Created user has a valid airlineId.

• Super Admin can update Airline Admin profile data.

• Super Admin can deactivate Airline Admin account.

• Deactivated Airline Admin cannot log in or access protected endpoints.

F2.4 Customer Account Management

Module: identity.user

Endpoints:

• GET /api/users/customers

• GET /api/users/customers/{customerId}

• PATCH /api/users/customers/{customerId}/deactivate

Access: SUPER_ADMIN

Purpose: Allow Super Admin to view and deactivate customer accounts.

Expected Classes

• CustomerResponse
• DeactivateUserResponse
• UserController or CustomerController under identity.user
• UserService

Relevant Business Rules

• Only SUPER_ADMIN can view all customer accounts.

• Only SUPER_ADMIN can deactivate customer accounts.

• Customers are Users with role CUSTOMER.

• Customer accounts are deactivated instead of permanently deleted.

• Deactivated customers must not access protected endpoints.

• Historical bookings, payments, refunds, tickets, notifications, and audit records must remain stored.

Implementation Notes

• Customer listing filters users by role CUSTOMER.

• Deactivate changes userStatus to DEACTIVATED.

• The service should prevent accidental deactivation of non-customer users through customer-specific endpoints.

Done When

• Super Admin can list customers.

• Super Admin can view customer details.

• Super Admin can deactivate a customer.

• Customer is not deleted from the database.

• Non-Super Admin users cannot access these endpoints.

Phase 3 — Catalog

Goal: Implement airports, flights, seats, and public flight search. Catalog must be implemented before reservation because booking depends on existing flights and available seats.

F3.1 Airport Management

Module: catalog.airport

Endpoints:

• POST /api/airports

• PUT /api/airports/{airportId}

• GET /api/airports

• GET /api/airports/{airportId}

Access: POST / PUT: SUPER_ADMIN; GET: Public

Purpose: Allow Super Admin to create/update airports and allow public users to list and view airports for flight search.

Expected Classes

• Airport

• AirportRepository

• AirportService

• AirportController

• CreateAirportRequest

• UpdateAirportRequest

• AirportResponse

Relevant Business Rules

• Only SUPER_ADMIN can create or update airports.

• Airport list and airport details are public because they are needed for flight search.

• Airport code should be unique.

• Airport data should include airport code, airport name, city, and country.

• Flights use airports as origin and destination, so an airport should not be deleted if referenced by flights.

Implementation Notes

• Use unique constraint on airportCode.

• Use request validation for airportCode, airportName, city, and country.

• If deletion is ever needed later, prefer deactivation/status instead of deleting referenced data.

Done When

• Super Admin can create airports.

• Super Admin can update airports.

• Public users can list airports.

• Public users can view airport details.

• Duplicate airport code returns 409 Conflict.

F3.2 Flight Management

Module: catalog.flight

Endpoints:

• POST /api/flights

• PUT /api/flights/{flightId}

• PATCH /api/flights/{flightId}/cancel

• GET /api/flights

Access: AIRLINE_ADMIN for create/update/cancel; AIRLINE_ADMIN and SUPER_ADMIN for listing according to access rules

Purpose: Allow Airline Admins to manage flights for their assigned airline and allow authorized admins to list flights.

Expected Classes

• Flight

• FlightStatus

• FlightRepository

• FlightService

• FlightController

• CreateFlightRequest

• UpdateFlightRequest

• FlightResponse

Relevant Business Rules

• Airline Admin can add flights only for their assigned airline.

• Airline Admin cannot update or cancel flights belonging to another airline.

• Each flight must include route, schedule, price, seat capacity, and availability information.

• The origin airport and destination airport must exist.

• Origin and destination airports should not be the same.

• The assigned airline must be active when creating a new flight.

• Inactive airline cannot have new flights added.

• Cancelled flights remain stored for history and reports.

• When flight information is updated, affected customers should be notified later through notification logic.

• When a flight is cancelled, customers with bookings on that flight should be notified later.

Implementation Notes

• AirlineId is taken from the logged-in Airline Admin principal, not from the request body.

• Super Admin listing may show all flights; Airline Admin listing must be restricted to assigned airline.

• Flight cancellation changes flightStatus to CANCELLED, not delete the row.

Done When

• Airline Admin can create a flight for their assigned airline.

• Airline Admin can update only their airline flights.

• Airline Admin can cancel only their airline flights.

• Flight status is handled correctly.

• Unauthorized airline access returns 403 Forbidden.

F3.3 Seat Management

Module: catalog.seat

Endpoints:

• POST /api/flights/{flightId}/seats

• PUT /api/seats/{seatId}

• GET /api/flights/{flightId}/seats

• GET /api/flights/{flightId}/seats/available

Access: POST / PUT: AIRLINE_ADMIN; GET: Public

Purpose: Allow Airline Admins to manage seats for their assigned airline flights and allow public users to view seat availability.

Expected Classes

• Seat

• CabinClass

• SeatRepository

• SeatService

• SeatController

• CreateSeatsRequest

• UpdateSeatRequest

• SeatResponse

Relevant Business Rules

• Each seat belongs to one flight.

• Airline Admin can create or update seats only for flights belonging to their assigned airline.

• Each seat number must be unique within the same flight.

• Each seat can be assigned to only one passenger.

• Seat availability must be checked before booking.

• Public users can view flight seats and available seats before booking.

• Seat availability must remain consistent when multiple customers try to book at the same time.

Implementation Notes

• Use a unique constraint on flightId + seatNumber.

• Availability can be derived from whether a seat is assigned to a passenger or stored explicitly if needed.

• Concurrency rules should be handled carefully when implementing reservation.

Done When

• Airline Admin can create seats for a flight.

• Airline Admin can update seat information.

• Public users can view all seats for a flight.

• Public users can view only available seats.

• Duplicate seat number for the same flight returns 409 Conflict.

F3.4 Public Flight Search and Details

Module: catalog.flight

Endpoints:

• GET /api/flights/search

• GET /api/flights/{flightId}

Access: Public

Purpose: Allow users to search available flights and view flight details before booking.

Relevant Business Rules

• Flight search is public and does not require login.

• Users can search by origin, destination, departure date, passenger count, and availability.

• Search results must show only relevant available/scheduled flights according to filters.

• Search results must reflect current flight availability.

• Flight details should include route, schedule, airline, price, seat capacity, available seats, and flight status.

Implementation Notes

• Search should use indexes where needed for origin, destination, date, and status.

• Available seats can be calculated from seat capacity and assigned seats or from seat records.

• This feature starts the customer end-to-end workflow.

Done When

- Users can search flights by origin, destination, date, and passenger count.
- Users can view detailed flight information.
- Search results reflect current availability.
- Public users can access these endpoints without JWT.

Phase 4 — Reservation

Goal: Implement booking creation, passenger details, seat selection, booking views, and cancellation. Reservation starts after Identity and Catalog because booking requires a logged-in customer, an available flight, and available seats.

F4.1 Create Booking

Module: reservation.booking

Endpoint: POST /api/bookings

Access: CUSTOMER

Purpose: Create a booking for a selected flight with PENDING_PAYMENT status.

Expected Classes

• Booking

• BookingStatus

• BookingRepository

• BookingService

• BookingController

• CreateBookingRequest

• BookingResponse

Relevant Business Rules

• Only CUSTOMER can create a booking.

• Booking can be created only for an available flight.

• Booking is created with PENDING_PAYMENT status before payment confirmation.

• The booking must be linked to the logged-in customer.

• The booking must be linked to the selected flight.

• Total amount is calculated based on flight base price and passenger count.

• Passenger count must be positive and must not exceed available seats.

• A ticket cannot be generated for a booking that is not confirmed.

Implementation Notes

• CustomerId comes from the authenticated user principal, not from the request body.

• BookingReference should be generated by the backend.

• Seat assignment happens in the next feature when passengers are added.

Done When

• Customer can create a booking.

• Booking status starts as PENDING_PAYMENT.

• Booking is linked to customer and flight.

• Total amount is calculated correctly.

• Unauthorized users cannot create bookings.

F4.2 Add Passengers and Select Seats

Module: reservation.passenger + reservation.booking

Endpoint: POST /api/bookings/{bookingId}/passengers

Access: CUSTOMER

Purpose: Add passenger details to a booking and assign one available seat to each passenger.

Expected Classes

• Passenger

• PassengerRepository

• PassengerService

• AddPassengersRequest

• PassengerResponse

Relevant Business Rules

• Only the customer who owns the booking can add passengers to it.

• A booking can contain one or more passenger records.

• Passenger does not have to be a registered user.

• Passenger details must be added during the booking process.

• Each passenger must have one selected available seat.

• A selected seat cannot be assigned to another passenger.

• Seats must belong to the same flight as the booking.

• Seat availability must be checked before assignment.

• The number of passengers added should match the booking passenger count.

Implementation Notes

• The customer who creates the booking is responsible for entering correct passenger details.

• Use transaction boundaries to keep passenger creation and seat assignment consistent.

• Concurrency handling becomes important when multiple customers try to select seats.

Done When

• Customer can add passenger details.

• Customer can select seats.

• Already assigned seats cannot be selected again.

• Seat and passenger records are saved consistently.

• Customer cannot add passengers to another customer’s booking.

F4.3 View Bookings

Module: reservation.booking

Endpoints:

• GET /api/bookings/my

• GET /api/bookings/{bookingId}

• GET /api/bookings/airline

• GET /api/bookings

Access: CUSTOMER, AIRLINE_ADMIN, SUPER_ADMIN according to endpoint and ownership rules

Purpose: Allow users to view bookings according to their role and ownership restrictions.

Relevant Business Rules

• Customers can access only their own bookings.

• Customers cannot access bookings that do not belong to them.

• Airline Admins can view only bookings related to their assigned airline.

• Airline Admins can view passengers for flights belonging to their assigned airline only.

• Super Admin can view all bookings across the system.

• Cancelled bookings remain stored and can still be viewed for history and reports.

Implementation Notes

• Access checks are done in service methods in addition to role-based security.

• GET /api/bookings/{bookingId} can be accessed by multiple roles, but ownership/airline checks must be applied.

• Pagination and filtering should be used for list endpoints.

Done When

• Customer can view own bookings.

• Customer cannot view another customer’s bookings.

• Airline Admin can view assigned airline bookings only.

• Super Admin can view all bookings.

• Forbidden access returns 403 Forbidden.

F4.4 Cancel Booking

Module: reservation.booking

Endpoint: PATCH /api/bookings/{bookingId}/cancel

Access: CUSTOMER

Purpose: Allow a customer to cancel an eligible booking.

Relevant Business Rules

• Only the customer who owns the booking can cancel it through the customer cancellation endpoint.

• Booking can be cancelled only if eligible according to its status and timing rules.

• If booking is cancelled before payment, no refund is required.

• If a confirmed paid booking is cancelled, refund eligibility must be checked later in the refund flow.

• Cancelled booking status becomes CANCELLED.

• Cancelled bookings remain stored for history and reports.

• The system sends a cancellation email after booking cancellation when notification is implemented.

Implementation Notes

• Do not delete booking records.

• Cancellation may release seats depending on booking/payment status and business decision.

• Refund creation is not part of this feature; it belongs to billing.refund.

Done When

- Customer can cancel eligible booking.
- Booking status becomes CANCELLED.
- Historical booking record remains stored.
- Customer cannot cancel another customer’s booking.
- Ineligible cancellation returns a conflict or business error.

Phase 5 — Billing: Payment

Goal: Implement payment request creation, payment gateway callback, and payment status handling. Payment depends on booking because payment is created for an existing booking.

F5.1 Create Payment Request

Module: billing.payment

Endpoint: POST /api/payments

Access: CUSTOMER

Purpose: Create a payment request for an existing booking.

Expected Classes

• Payment

• PaymentStatus

• PaymentRepository

• PaymentService

• PaymentController

• CreatePaymentRequest

• PaymentResponse

Relevant Business Rules

• Only CUSTOMER can create a payment request for their own booking.

• Payment must be linked to an existing booking.

• A booking must be in a payable state such as PENDING_PAYMENT before payment creation.

• Payment status starts as PENDING.

• Each booking should have one payment record according to the ERD/API design.

• The system stores only necessary payment references, transaction IDs, payment tokens, payment amounts, and payment statuses.

• The system must not store sensitive card information.

Implementation Notes

• Payment gateway integration can be mocked in the first implementation.

• Payment amount should match the booking total amount.

• Duplicate payment for the same booking should be prevented.

Done When

• Customer can create a payment request.

• Payment is saved with PENDING status.

• Payment is linked to the booking.

• Payment amount matches booking total amount.

• Sensitive card information is not stored.

F5.2 Payment Gateway Callback

Module: billing.payment.gateway

Endpoint: POST /api/payments/callback

Access: PAYMENT_GATEWAY

Purpose: Receive the payment result from the payment gateway and update payment and booking statuses safely.

Relevant Business Rules

• Payment gateway callback is a special protected endpoint, not a normal frontend user endpoint.

• Gateway response must be verified before updating payment, booking, or refund statuses.

• Payment status is updated based on the payment gateway response.

• If payment succeeds, payment status becomes SUCCESS.

• If payment succeeds, booking status becomes CONFIRMED.

• If payment fails, payment status becomes FAILED.

• If payment fails, booking is not confirmed and no ticket is generated.

• Payment and refund results should be logged for audit and troubleshooting later.

Implementation Notes

• Use X-Gateway-Signature or a mock signature validation during development.

• Callback handling should be idempotent where possible to avoid double updates.

• Ticket generation can be called directly after successful payment in the first version.

Done When

• Payment callback updates payment status.

• Successful payment confirms booking.

• Failed payment does not generate a ticket.

• Invalid gateway signature is rejected.

• Payment result is handled safely without corrupting booking status.

F5.3 View Payment Data

Module: billing.payment

Endpoints:

• GET /api/payments/{paymentId}

• GET /api/bookings/{bookingId}/payment

• GET /api/payments/my

Access: CUSTOMER and SUPER_ADMIN where applicable

Purpose: Allow authorized users to view payment data according to ownership and role rules.

Relevant Business Rules

• Customers can view only their own payment records.

• Customers must not access payment records related to another customer's booking.

• Super Admin can view payment details when the API Contract allows it.

• Payment-related records must be protected from unauthorized users.

• Sensitive card information must never be returned in responses.

Implementation Notes

• Ownership is checked through Payment -> Booking -> Customer.

• Use role-based security plus service-level ownership checks.

• Paginated responses should be used for list endpoints such as my payments.

Done When

• Customer can view own payment records.

• Customer cannot view another customer's payment records.

• Payment data is protected from unauthorized users.

• Responses do not expose sensitive card data.

Phase 6 — Ticketing

Goal: Generate and manage tickets after successful booking confirmation. Ticket generation depends on successful payment and confirmed booking.

F6.1 Generate Ticket

Module: ticketing

Endpoint: POST /api/bookings/{bookingId}/tickets/generate

Access: SYSTEM_INTERNAL

Purpose: Generate tickets after successful payment confirmation and booking confirmation.

Expected Classes

• Ticket

• TicketStatus

• TicketRepository

• TicketService

• TicketController

• TicketResponse

Relevant Business Rules

• Ticket is generated only after booking confirmation.

• Ticket is linked to a confirmed booking.

• Ticket is linked to a passenger.

• A ticket cannot be generated for a booking that is not confirmed.

• Ticket status starts as ISSUED.

• Ticket records remain stored and are not deleted.

• Ticket details are sent to the customer by email after ticket generation when notification is implemented.

Implementation Notes

• This endpoint is internal and should not be called directly by normal frontend users.

• In the first implementation, PaymentService can call TicketService directly after successful payment.

• Prevent duplicate tickets for the same passenger.

Done When

• Ticket is generated after successful payment confirmation.

• Ticket status is ISSUED.

• Ticket is linked to booking and passenger.

• Ticket is not generated for unconfirmed bookings.

• Duplicate ticket for the same passenger is prevented.

F6.2 View Tickets

Module: ticketing

Endpoints:

• GET /api/tickets/my

• GET /api/tickets/{ticketId}

• GET /api/bookings/{bookingId}/tickets

Access: CUSTOMER

Purpose: Allow customers to view their own tickets after booking confirmation and ticket generation.

Relevant Business Rules

• Customer can view tickets only after the related booking is confirmed.

• Customers can view only their own tickets.

• Customers must not access tickets related to another customer's booking.

• Ticket records remain stored for history and reports.

Implementation Notes

• Ownership is checked through Ticket -> Booking -> Customer.

• GET /api/bookings/{bookingId}/tickets must verify booking ownership before returning tickets.

Done When

• Customer can view own tickets.

• Customer cannot view another customer's tickets.

• Ticket details are returned only for authorized customer.

F6.3 Cancel Ticket

Module: ticketing

Endpoint: PATCH /api/tickets/{ticketId}/cancel

Access: SYSTEM_INTERNAL or SUPER_ADMIN

Purpose: Cancel or invalidate a ticket when the related booking is cancelled or when a Super Admin performs an authorized cancellation.

Relevant Business Rules

• Ticket can be cancelled when the related booking is cancelled.

• Ticket status becomes CANCELLED.

• Cancelled ticket records remain stored and are not deleted.

• Normal customers should not directly cancel tickets through this endpoint unless the API Contract is changed.

• Ticket cancellation must stay consistent with booking cancellation.

Implementation Notes

• Booking cancellation can trigger ticket cancellation internally.

• Super Admin access may be allowed for operational correction according to the API Contract.

Done When

- Ticket can be cancelled when related booking is cancelled.
- Ticket status becomes CANCELLED.
- Ticket record remains stored.
- Unauthorized users cannot cancel tickets.

Phase 7 — Notification

Goal: Save and send notifications for booking confirmation, ticket email, cancellation, refund, and flight updates. Notification failure must not cancel booking or payment operations.

F7.1 Save Notification Records

Module: notification

Purpose: Save notification records in the database so the system can track sent and failed notifications.

Expected Classes

• Notification

• NotificationType

• NotificationStatus

• NotificationRepository

• NotificationService

• NotificationController

Relevant Business Rules

• The system sends booking confirmation notification after booking is confirmed.

• The system sends ticket details to the customer by email after ticket generation.

• The system sends cancellation notification after booking cancellation.

• The system sends refund notification after refund processing.

• The system sends flight update notifications to affected customers when flight information changes or a flight is cancelled.

• Notification failure must not cancel booking, payment, ticketing, or refund operations.

• Notification status can be PENDING, SENT, or FAILED.

Implementation Notes

• This feature can be implemented before real email sending by saving notification records only.

• Notification records should link to the user and optionally to the booking when applicable.

• Actual email sending can be mocked at first.

Done When

• Notification records are saved in the database.

• Notification type is stored correctly.

• Notification status can be PENDING, SENT, or FAILED.

• Notification failure does not break core business flow.

F7.2 Send Notification / Mock Email

Module: notification.email

Endpoint: POST /api/notifications/send

Access: SYSTEM_INTERNAL

Purpose: Send or mock email notifications for system events.

Relevant Business Rules

• This endpoint is used internally by the system, not by normal frontend users.

• The system should support sending booking confirmation email.

• The system should support sending ticket email.

• The system should support sending cancellation email.

• The system should support sending refund email.

• The system should support sending flight update email.

• Failed notification attempts must be recorded safely.

Implementation Notes

• Use a mock email sender in the first implementation.

• Update notification status to SENT after successful mock/send.

• Update notification status to FAILED if sending fails.

Done When

• System can send or mock booking confirmation email.

• System can send or mock ticket email.

• System can send or mock cancellation email.

• System can mark failed notification as FAILED.

• Failed notification does not break booking or payment flow.

F7.3 View and Retry Notifications

Module: notification

Endpoints:

• GET /api/notifications/my

• GET /api/notifications/{notificationId}

• GET /api/notifications

• POST /api/notifications/{notificationId}/retry

Access: CUSTOMER and SUPER_ADMIN according to endpoint

Purpose: Allow customers to view their notifications and allow Super Admin to view all notifications and retry failed ones.

Relevant Business Rules

• Customers can view only their own notifications.

• Customers must not access another user's notifications.

• Super Admin can view all notifications.

• Super Admin can retry failed notifications.

• Retry should be allowed only for failed or pending notifications depending on the final business decision.

Implementation Notes

• Ownership is checked through notification.userId.

• Retry calls the notification email service again and updates the status.

Done When

• Customer can view own notifications.

• Customer cannot view another user’s notifications.

• Super Admin can view all notifications.

• Super Admin can retry failed notifications.

Phase 8 — Billing: Refund

Goal: Implement refund request, refund gateway callback, and refund status tracking. Refund belongs under billing.refund and depends on payment and cancellation because refunds apply only to eligible paid bookings.

F8.1 Request Refund

Module: billing.refund

Endpoint: POST /api/refunds

Access: CUSTOMER

Purpose: Allow a customer to request a refund for an eligible paid and cancelled booking.

Expected Classes

• Refund

• RefundStatus

• RefundRepository

• RefundService

• RefundController

• CreateRefundRequest

• RefundResponse

Relevant Business Rules

• Customer can request a refund only for eligible paid bookings.

• Refund cannot be requested for an unpaid booking.

• Refund is linked to payment, not directly to booking.

• The customer must own the booking related to the payment.

• Refund status starts as PENDING.

• A payment should have at most one refund request according to the ERD/design.

• Refund processing is handled through the payment gateway.

• The system sends refund notification after refund result is processed when notification is implemented.

Implementation Notes

• Booking can be reached through Refund -> Payment -> Booking.

• No refund record is needed if no refund was requested.

• Eligibility rules can start simple and become more detailed later.

Done When

• Customer can request refund for eligible paid booking.

• Refund status starts as PENDING.

• Refund cannot be created for unpaid booking.

• Refund cannot be requested by a customer who does not own the booking.

• Duplicate refund for the same payment is prevented.

F8.2 Refund Gateway Callback

Module: billing.refund

Endpoint: POST /api/refunds/callback

Access: PAYMENT_GATEWAY

Purpose: Receive refund result from the payment gateway and update refund status safely.

Relevant Business Rules

• Refund gateway callback is a special protected endpoint, not a normal frontend user endpoint.

• Gateway response must be verified before updating refund status.

• Refund status is updated based on gateway response.

• Successful refund becomes REFUNDED.

• Failed refund becomes FAILED.

• Failed refunds are recorded and handled safely.

• Payment and refund results should be logged for audit and troubleshooting.

Implementation Notes

• Use X-Gateway-Signature or a mock signature validation during development.

• Callback handling should be idempotent where possible.

Done When

• Refund callback updates refund status.

• Successful refund becomes REFUNDED.

• Failed refund becomes FAILED.

• Invalid gateway signature is rejected.

• Refund result is handled safely.

F8.3 View Refund Data

Module: billing.refund

Endpoints:

• GET /api/refunds/{refundId}

• GET /api/bookings/{bookingId}/refund

• GET /api/refunds/my

• GET /api/refunds

Access: CUSTOMER and SUPER_ADMIN according to endpoint

Purpose: Allow authorized users to view refund data according to ownership and role rules.

Relevant Business Rules

• Customers can view only their own refund requests.

• Customers must not access another customer’s refund data.

• Super Admin can view all refund requests.

• Refund data is protected from unauthorized users.

• Reports can use historical refund data.

Implementation Notes

• Ownership is checked through Refund -> Payment -> Booking -> Customer.

• GET /api/bookings/{bookingId}/refund should verify booking ownership or Super Admin access.

Done When

• Customer can view own refund requests.

• Customer cannot view another customer’s refund requests.

• Super Admin can view all refund requests.

• Refund data is protected from unauthorized users.

Phase 9 — Reporting and Audit

Goal: Implement system reports and audit logs after the main business flows are working. Reports depend on existing booking, payment, refund, airline, and customer data. Audit logs depend on sensitive admin and financial actions.

F9.1 Audit Logs

Module: reporting.audit

Endpoints:

• GET /api/audit-logs

• GET /api/audit-logs/{auditLogId}

Access: SUPER_ADMIN

Purpose: Track sensitive system actions for audit, history, and troubleshooting.

Expected Classes

• AuditLog

• AuditLogRepository

• AuditLogService

• AuditLogController

• AuditLogResponse

Relevant Business Rules

• Sensitive admin actions must be logged.

• Payment and refund results must be logged.

• Booking creation, cancellation, payment results, and refund results are kept for reports and troubleshooting.

• Only SUPER_ADMIN can view audit logs.

• Audit logs should not be deleted as part of normal operations.

Actions to Log

• CREATE_AIRLINE_ADMIN

• DEACTIVATE_AIRLINE

• DEACTIVATE_AIRLINE_ADMIN

• DEACTIVATE_CUSTOMER

• PAYMENT_RESULT

• REFUND_RESULT

• BOOKING_CANCELLATION

Implementation Notes

• Audit logging can be introduced gradually as sensitive actions are implemented.

• Audit log should store actor userId, action type, target type, target id, timestamp, and useful details.

Done When

• Sensitive admin actions are logged.

• Payment and refund results are logged.

• Super Admin can view audit logs.

• Non-Super Admin users cannot access audit logs.

F9.2 Booking Report

Module: reporting.report

Endpoint: GET /api/reports/bookings

Access: AIRLINE_ADMIN and SUPER_ADMIN

Purpose: Provide booking report data according to user role.

Relevant Business Rules

• Super Admin can view all booking reports.

• Airline Admin can view only booking reports for their assigned airline.

• Reports can use historical booking, airline, customer, payment, and refund data.

• Report data must respect role-based access control.

Implementation Notes

• Airline Admin filtering should be based on assigned airlineId from the authenticated user.

• Super Admin may filter by airline, status, and date if supported.

Done When

• Super Admin can view all booking report data.

• Airline Admin can view only assigned airline booking report data.

• Unauthorized users cannot access booking reports.

F9.3 Payment Report

Module: reporting.report

Endpoint: GET /api/reports/payments

Access: SUPER_ADMIN

Purpose: Provide payment report data for Super Admin.

Relevant Business Rules

• Only SUPER_ADMIN can view payment reports.

• Payment reports can use historical payment, booking, airline, and customer data.

• Payment report data should support filtering by date, status, and payment method if implemented.

• Sensitive card data must not appear in reports.

Done When

• Super Admin can view payment report data.

• Report can filter by date, status, and payment method when supported.

• Sensitive payment card data is not exposed.

F9.4 Refund Report

Module: reporting.report

Endpoint: GET /api/reports/refunds

Access: SUPER_ADMIN

Purpose: Provide refund report data for Super Admin.

Relevant Business Rules

• Only SUPER_ADMIN can view refund reports.

• Refund reports can use historical refund, payment, booking, airline, and customer data.

• Refund report data should support filtering by date, status, and airline if implemented.

• Failed refunds should remain recorded for troubleshooting.

Done When

• Super Admin can view refund report data.

• Report can filter by date, status, and airline when supported.

• Failed refunds appear in report data for troubleshooting.

Sprint Plan

Sprint 1 — Identity Basics

• UserRole enum

• UserStatus enum

• User entity

• UserRepository

• Register Customer API

• Login API

Sprint 2 — JWT and Current User

• JWT generation

• JWT validation filter

• SecurityConfig

• GET /api/auth/me

• PUT /api/auth/me

• PUT /api/auth/change-password

Sprint 3 — Administration Setup

• Create initial Super Admin

• Create Airline

• Update Airline

• Deactivate Airline

• Create Airline Admin

• Update Airline Admin

• Deactivate Airline Admin

• Customer account management

Sprint 4 — Catalog

• Create Airport

• Update Airport

• Get Airports

• Create Flight

• Update Flight

• Cancel Flight

• Create Seats

• Get Available Seats

• Search Flights

• Get Flight Details

Sprint 5 — Reservation

• Create Booking

• Add Passengers

• Select Seats

• View My Bookings

• View Booking Details

• Cancel Booking

Sprint 6 — Payment and Ticketing

• Create Payment Request

• Mock Payment Callback

• Confirm Booking after successful payment

• Generate Ticket

• View Tickets

Sprint 7 — Notification and Refund

• Save Notification Records

• Mock Email Sending

• Retry Failed Notification

• Request Refund

• Mock Refund Callback

• View Refunds

Sprint 8 — Reports and Audit

• Audit Logs

• Booking Report

• Payment Report

• Refund Report

Definition of Done for Every Feature

• The endpoint matches the API Contract.

• Request DTO and Response DTO are created when needed.

• Validation is applied to request DTOs.

• Relevant business rules are implemented in the correct layer.

• Entity and repository logic work correctly.

• The response follows the standard API response format.

• Failure responses are handled clearly.

• The endpoint is tested using Postman.

• At least one basic test is added where possible.

• Code is committed to Git with a clear commit message.

DOCKER , TESTING